AI Model Deployment & MLOps Assignment Report:

Objective: To deploy a sample AI model using Docker & Kubernetes and expose it via an API endpoint.

Steps Performed:

This document outlines the steps taken to complete the AI Model Deployment & MLOps assignment, as per the task description.

- Choosing a Sample AI Model: A simple Logistic Regression model was trained on the Iris dataset using the scikit-learn library in Python. This model was chosen for its simplicity and ease of containerization. The training script (model.py) saved the trained model to a file named iris_model.joblib.

- Creating the Dockerfile: A Dockerfile was created to define the environment for running the API. This file specified the base Python image, set the working directory, copied the necessary files (requirements.txt, iris_model.joblib, and app.py), installed the required Python libraries, exposed port 80, and defined the command to run the Flask API.

- Developing the API: A Python web API was developed using the Flask framework (app.py). This API loads the trained iris_model.joblib file and exposes a POST endpoint at /. This endpoint accepts JSON data containing the four features of an Iris flower and returns the model's prediction as a JSON response. A requirements.txt file was created to list the dependencies for the API.

- Building the Docker Image: The Docker image was built using the command docker build -t ai-model-api. from the directory containing the Dockerfile. This command created a container image named ai-model-api that encapsulates the API code, the trained model, and all necessary dependencies.

- Tagging the Docker Image: The Docker image was tagged with the user's Docker Hub username and the latest tag using the command docker tag ai-model-api your-dockerhub-username/ai-model-api:latest. This step prepares the image for pushing to a container registry.

- Pushing the Docker Image to Docker Hub: The tagged Docker image was pushed to Docker Hub using the command docker push your-dockerhub-username/ai-model-api:latest. This makes the container image accessible to the Kubernetes cluster.

- Creating the Kubernetes Deployment YAML: A deployment.yaml file was created to define how the Docker image should be deployed on the Kubernetes cluster. This file specified the number of replicas (set to 2), a selector to identify the Pods, and the container details, including the image name from Docker Hub and the port to expose (80).

- Creating the Kubernetes Service YAML: A service.yaml file was created to expose the deployed AI model API as a service within the Kubernetes cluster. The Service type was set

to LoadBalancer to make the API accessible externally. The file also defined the ports to be used (port 80 for both the service and the target Pods) and the selector to target the correct Pods.

- Starting Minikube: A local Kubernetes cluster was started using Minikube with the command minikube start. This provided the environment for deploying and testing the application.

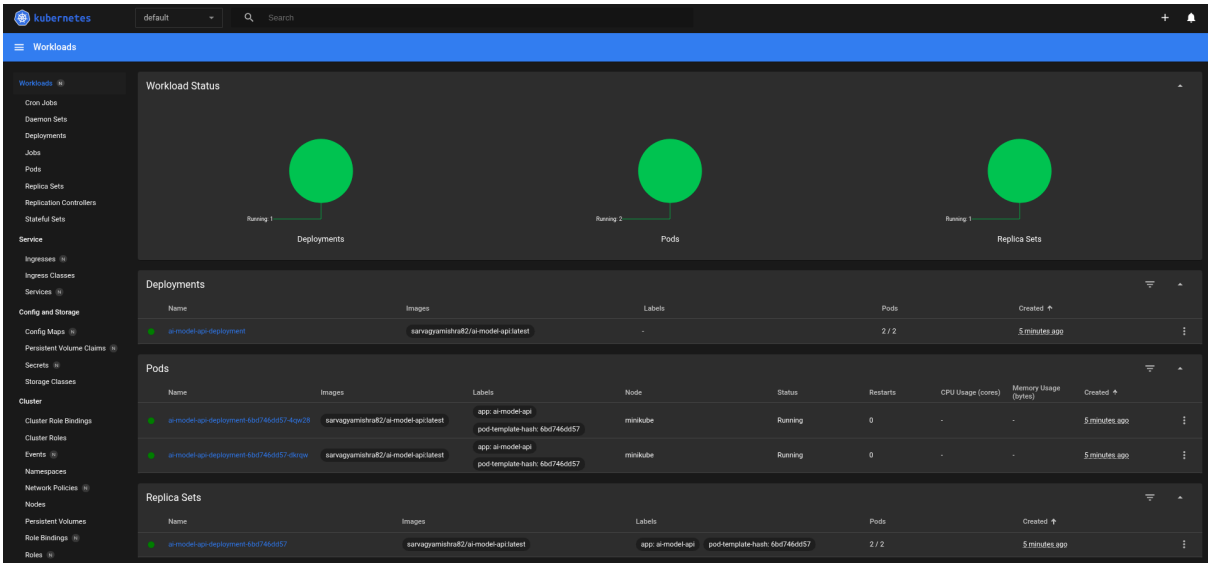
- Applying the Kubernetes Deployment and Service: The Kubernetes Deployment and Service were applied to the Minikube cluster using the kubectl apply -f deployment.yaml and kubectl apply -f service.yaml commands, respectively. This initiated the deployment of the AI model API.

- Verifying the Deployment and Service: The status of the Deployment and Service was verified using kubectl get deployments, kubectl get pods, and kubectl get services commands to ensure that the desired number of Pods were running and the Service was correctly exposed.

- Accessing and Testing the Deployed API: The API endpoint was accessed using the URL provided by Minikube (minikube service ai-model-api-service --url). The curl command was used to send a POST request with sample Iris flower data in JSON format to the API endpoint. A successful response containing the prediction ({"prediction": 0}) was received, confirming that the model was successfully deployed and functioning.

ScreenShot:

Minikube Dashboard-



Screenshot of the model running on Kubernetes